

Loop Engineering — von Prompts zu autonomen, selbstverifizierenden Schleifen

Quellen: [video-summary-HASGvvp1M3E.md](#), [video-summary-NeyVq965bOM.md](#), [video-summary-SFtiPOTLBHA.md](#), [video-summary-4NgKZerJpk8.md](#), [video-summary-LwhB_6VBwTQ.md](#), [video-summary-u2v17HBnhh8.md](#), [video-summary-RaraRJ0IZpA.md](#), [video-summary-qtte0zpnGks.md](#), [video-summary-TP73qyFWDcY.md](#), [ai-agent-workflow.md](#) (Punkt 8), [video-summary-zBI2bWO7RLc.md](#), [video-summary-JAszmnL5fyk.md](#)

"Loop Engineering" taucht in mindestens 10 Dateien dieses Repos auf, sechs der 17 im August-2026-Batch angesehenen Videos sind ihm sogar vollständig gewidmet — beim letzten [notes-audit-report.md](#) galt es deshalb erstmals als klarer Kandidat für einen eigenen Übersichtsartikel (im Audit vom 2026-08-01 war es das noch nicht: damals reichte die Erwähnung in `ai-agent-workflow.md` aus). Dieser Artikel bündelt das Konzept, das bisher über viele Einzel-Zusammenfassungen verstreut war.

Herkunft des Begriffs

Der Begriff geht auf zwei unabhängige, Anfang Juni 2026 fast zeitgleich viral gegangene Zitate zurück:

- **Boris Cherny** (Erfinder von Claude Code, @bcherny): *"Two of the most powerful features in Claude Code: /loop and /schedule. [...] I don't prompt Claude anymore [...] my job is to write loops."* (Tweet vom 30.03.2026, per WebSearch bestätigt)
- **Peter Steinberger** (OpenClaw, @steipete): *"Here's your monthly reminder that you shouldn't be prompting coding agents anymore. You should be designing loops that prompt your agents."* (Tweet vom 7./8. Juni 2026, per WebSearch bestätigt, laut zwei Videos mit 6,5–8,4 Mio. Aufrufen)

Andrew Ng widmete dem Prinzip Ende Juni 2026 (im Repo unterschiedlich mit "26." bzw. "~30." Juni datiert — nicht abschließend geklärt, siehe unten) einen eigenen Beitrag in seinem Newsletter "The Batch" (DeepLearning.AI) — ein Indiz dafür, dass mehr dahintersteckt als ein kurzlebiger Trend.

Die Grundidee: "Ihr wart der Loop"

Vor Loop Engineering musste der Mensch selbst die fehlende Feedback-Schleife eines Modells schließen: prompten → Antwort bekommen → kontrollieren → erneut prompten — Prompt Engineering war im Kern ein Ersatz dafür, dass das Modell sich nicht selbst verifizieren konnte. Zwei technische Verschiebungen lösten das auf ([video-summary-NeyVq965bOM.md](#)):

1. **"Agenten haben Hände bekommen"** — Modelle führen Code selbst aus, sehen Fehler direkt, lesen Dateien selbst
2. Modelle wurden fähig genug, bei Misserfolg **die Strategie zu wechseln statt denselben Fehler zu wiederholen**

Ein Loop ist damit keine bessere Prompt-Formulierung, sondern die Gestaltung des gesamten Ablaufs, in dem eine KI arbeitet: wie oft sie etwas durchgeht, woran sie das Ergebnis misst, woran sie erkennt, dass sie fertig ist ([video-summary-LwhB_6VBwTQ.md](#)).

Jargon-freier Einstieg für Einsteiger: [video-summary-JAszmnL5fyk.md](#) erklärt dasselbe Grundprinzip ohne Fachbegriffe über ein einfaches Drei-Stufen-Modell (Chatbot → Workflow → Agent: "beim Workflow entscheidet der Mensch die Schritte, beim Agenten die KI selbst") plus einen "Planen-Handeln-Prüfen"-Loop, der sich inhaltlich mit dem "inneren

Loop" oben deckt — guter Zubringer für Leser, die zuerst dieses Konzept brauchen, bevor sie den Rest dieses Artikels lesen.

Einordnung in eine größere Stufenleiter — mit ungelöster Begriffsverwirrung

[video-summary-SFtiPOTLBHA.md](#) ordnet Loop Engineering als vierte von fünf aufeinander aufbauenden Schichten ein: **Prompt Engineering** (2022) → **Context Engineering** (Mitte 2025) → **Harness Engineering** (Anfang 2026) → **Loop Engineering** (Juni 2026) → **Graph Engineering** ("frisch aus dem Ofen", eher Steinbergers Scherz als etabliertes Konzept). Kernthese: Jede Stufe kompensiert eine aktuelle Modell-Schwäche, der Hebel wandert von reiner Prompt-Formulierung zu Kontext-Kuratierung, Tooling/Guardrails, Verifikationsschleifen und zuletzt explizit modelliertem Kontrollfluss. Belegt u. a. mit einem realen LangChain-Terminal-Bench-Sprung (52,8 % → 66,5 % Genauigkeit durch reines Harness Engineering, ohne Modellwechsel) und Googles ADK 2.0 als Graph-basierter Workflow-Engine.

[video-summary-4NgKZerJpk8.md](#) zeichnet dieselbe Begriffskette unabhängig, aber mit einer **anderen Container-Logik**: Dort umschließt der "Harness" bereits vier gleichrangige Bausteine (Prompt-/Context-/Tool-/Skill-Engineering) als Ganzes; ein Loop ist ein *innerhalb* eines Harness laufender, zielverfolgender Prozess; Graph Engineering verbindet mehrere Loop-plus-Harness-Einheiten (potenziell mit unterschiedlichen, kostenoptimierten Modellen wie Haiku/GPT/Kimi K3) zu einem Ökosystem.

Diese beiden Modelle widersprechen sich nicht in der Substanz, verwenden "Harness" aber strukturell unterschiedlich (Container vs. gleichrangiger Baustein) — eine dritte, wieder andere Lesart (Harness = das konkrete Werkzeug selbst, z. B. Claude Code) dokumentiert [claude-skills-ueberblick.md](#). Laut [notes-audit-report.md](#) (2026-08-22) ist das ein reiner Begriffs-, kein Sachverhalts-Widerspruch — die Begriffswelt rund um Agent-Engineering hat sich 2026 schlicht noch nicht gesetzt.

Eine dritte, unabhängige Quelle bestätigt zumindest die Kern-Definition von Graph Engineering: [video-summary-zBI2bWO7RLc.md](#) (2026-09-07) beschreibt ein schlankeres 3-Stufen-Modell (Prompt → Loop → Graph, ohne Context/Harness) mit derselben Definition wie SFtiPOTLBHA: "der Graph ersetzt die Loop nicht, er umschließt sie" — ein fester Blueprint aus Knoten/Kanten, der organisiert, wie Loops zusammenspielen. Kein Widerspruch, eher eine dritte, engere Variante derselben Begriffsfamilie.

Die vier Bausteine eines Loops

Die konkreteste strukturelle Zerlegung liefert [video-summary-NeyVq965bOM.md](#):

1. **Die Spec** — was gebaut werden soll
2. **Die Checkliste** — was "fertig" konkret bedeutet (laut Video der meist übersehene, aber wichtigste Teil)
3. **Der Inspektor** — im Idealfall ein *separates* Modell, das die Arbeit bewertet; der Ausführungs-Agent darf sich nicht selbst abnehmen ("Vier-Augen-Prinzip")
4. **Das Budget** — harte Obergrenze an Versuchen/Tokens, sonst lief ein Loop per Definition unendlich weiter

Zwei Loop-Typen mit derselben Vokabel: der **innere Loop** (versuchen → prüfen → ausbessern, endet selbst sobald der Inspektor zufrieden ist) und der **äußere Loop** (Daueroutine auf festem Zeitplan, liest/schreibt ein Logbuch, endet erst wenn der Mensch stoppt). Vier-Fragen-Checkliste, ob sich ein Loop überhaupt lohnt: Wiederholt sich die Aufgabe? Gibt es ein klares "fertig"? Sind ein paar Fehlversuche/Tokens verkraftbar? Hat der Agent die nötigen Werkzeuge? Grundsatz: **erst Skill, dann Loop** — ein Loop wiederholt nur, was dem Agenten bereits manuell beigebracht wurde.

Zweite, unabhängige Achse — offen vs. geschlossen: [video-summary-uEdSEF3XCmk.md](#) (derselbe Kanal wie oben) führt eine eigene, orthogonale Unterscheidung ein: **geschlossene Loops** haben einen klar begrenzten Zielumfang (z. B.

"diese eine Datei bereinigen"), **offene Loops** ein bewusst unbegrenztes Ziel (z. B. "finde neue Optimierungsmöglichkeiten"). Das ist nicht dasselbe wie innerer/äußerer Loop (der eine beschreibt Zeitstruktur, der andere Zielumfang) — beide Achsen widersprechen sich nicht, lassen sich kombinieren. Praktischer Beitrag desselben Videos: ein live gebauter 3-Agenten-Orchestrator (Builder/Scout/Growth-Agent) mit expliziten Gate-Bedingungen zwischen den Schritten ("Schritt 2 erst starten, wenn Datei X existiert") — ein direkt übertragbares Muster für eigene Multi-Agent-Setups.

Kontrollsatz vs. objektives Kriterium

Der praktisch wichtigste Einzelpunkt stammt aus [video-summary-LwhB_6VBwTQ.md](#): Ein Kontrollsatz ("Prüf das nochmal", "sei kritisch") sagt dem Modell, dass es prüfen soll — ein Kriterium sagt ihm, **woran**. Nur Kriterien lassen sich zuverlässig in eine Schleife packen:

Vager Kontrollsatz	Objektives Kriterium
"Kontrolliere den Text noch mal auf Länge"	unter 300 Zeichen
"Achte darauf, dass alles Wichtige drin ist"	Material genannt, Pflegehinweis am Ende
"Sei kritisch mit deinem eigenen Entwurf"	keine Superlative

Fehlt eine explizite **Abbruchbedingung**, hört ein Loop dann auf, wenn *er selbst* meint, dass es reicht — nicht wenn der Maßstab des Nutzers erfüllt ist. Ohne Cap droht eine Endlosschleife, die Tokens und Zeit verbrennt. Nicht alles gehört in eine Schleife: Alles objektiv Messbare kann als Kriterium rein; alles, wo Urteil, Geschmack oder Faktenwissen nötig ist (z. B. "passt zur Marke", Materialangaben), bleibt beim Menschen. Zusätzlicher Grund für kurze Loops: das Kontextfenster füllt sich mit jeder Runde, die Antwortqualität kann laut Video über mehrere Runden spürbar sinken.

Die Vertrauensleiter

Ebenfalls aus [video-summary-NeyVq965bOM.md](#), vier Eskalationsstufen (Analogie: neuer Mitarbeiter bekommt am Tag eins nicht die Firmenkreditkarte):

1. **Rundbasiert** — normaler Chat, jeder Prompt einzeln freigegeben
2. **Zielbasiert** (`/goal`) — läuft bis zur Ziellinie durch, Mensch sieht nur das Endergebnis
3. **Zeitbasiert** (`/loop` , `/schedule`) — läuft nach festem Zeitplan, Mensch überfliegt Ergebnisse
4. **Proaktiv** — Agent sucht sich Arbeit selbst, höchste Autonomie und höchstes Risiko

Ausdrückliche Warnung: eine Stufe nach der anderen erklimmen, nicht direkt bei Stufe 4 einsteigen — "Vertrauen ist, was sich der Loop verdienen muss." [video-summary-u2v17HBnhh8.md](#) bestätigt dieselbe Zweiteilung praktisch: `/loop` ist zeitbasiert (Laufzeit vorgeben, kein inhaltliches Ende), `/goal` ist ergebnisbasiert (läuft bis eine explizit definierte Bedingung erfüllt ist).

In der Praxis: zwei vollständige `/goal` -Durchläufe

[video-summary-u2v17HBnhh8.md](#) zeigt, statt nur Kurzbeispiele, zwei komplett am Bildschirm mitverfolgte Läufe:

- **Website-QA**: 12 objektiv prüfbare Kriterien (SEO, Technik, Barrierefreiheit, Mobil). Claude schrieb dafür selbst ein Bash-Prüfskript `check.sh` , testete es gegen eine bekannte gute Vergleichsdatei, fand und behob dabei zwei

eigene Bugs im Skript — erst danach lief der eigentliche `/goal` -Lauf, der die vier bewusst eingebauten Fehler in der Zielfeile eigenständig fand und korrigierte.

- **Excel-Datenbereinigung:** 20 Zeilen mit absichtlichen Rechtschreibfehlern, batchweise in 5er-Gruppen korrigiert, mit fortlaufendem Korrektur-Log ("falsch → richtig" pro Zeile) und Status-Spalte — Laufzeit ca. 5 Minuten ohne Eingriff.

`/goal` ist dabei kein reiner Marketingbegriff: Per WebSearch bestätigt, prüft nach jedem Turn ein separates, kleineres Modell (Standard: Haiku), ob die Zielbedingung "erfüllt", "noch nicht erfüllt" oder "unmöglich" ist (offizielle Doku: code.claude.com/docs/en/goal).

Skalierung: von einem Loop zu vielen

[video-summary-RaraRJ0IZpA.md](#) kombiniert zwei Bausteine zu einer "Assembly Line": eine dreischichtige Wissensbasis nach Andrej Karpathys **LLM-Wiki**-Pattern (RAW/Wiki/Schema — deckt sich mit zwei weiteren, unabhängig im Repo dokumentierten Umsetzungen desselben Musters) als Gedächtnis, plus eine Pipeline aus automatisch generierten Arbeitspaketen: pro Paket ein **Bau-Agent**, ein unabhängiger **Prüf-Agent** mit Nachbesserungsschleife, gebündelt in sequenziellen "Wellen". In der gezeigten Demo lief ein Auftrag mit 24 Arbeitspaketen fast zwei Stunden komplett unbeaufsichtigt — das Video thematisiert an keiner Stelle, ob/wo ein Mensch vor Abschluss eingreifen müsste (Governance-Lücke, siehe unten).

[video-summary-4NgKZerJpk8.md](#) beschreibt dieselbe Skalierungsidee als "Graph Engineering": mehrere Loops parallel, mit geteiltem Wissen, idealerweise je eigenem (günstigerem oder passenderem) Modell — Beispielskette Research-Loop → Bau-Loop → Review-Loop, als direkte Kostenoptimierung gegenüber einem einzigen Dauer-Loop mit durchgehend teurem Modell.

Governance-Lücke: keine der Quellen behandelt das systematisch

Mehrstündige, unbeaufsichtigte Loop-Läufe mit vollem Dateisystemzugriff werfen dieselbe Frage auf, die [video-summary-sQBinJA_zxU.md](#) unter "Human in the Loop bei kritischen Aktionen" bereits stellt. Für sicherheits- oder kundenrelevante Aufgaben deckt sich das mit der bereits bestehenden Regel in [ki-guidelines-hardware-unit.md](#) (Punkt 2: KI-Entwurf ist nie finale Freigabe) — für interne, unkritische Großaufträge (dort Punkt 5) ist die Assembly-Line-Struktur mit eingebautem Prüf-Agent dagegen plausibel übertragbar.

Praktische Einordnung

Für Team-/Automatisierungs-Entscheidungen sind vier Muster aus diesem Themenblock direkt einsetzbar, unabhängig von der Begriffsdebatte um Harness/Graph:

1. **Objektive Kriterien statt Kontrollsätze formulieren** — der einzige Hebel, der eine Schleife überhaupt zuverlässig macht.
2. **Immer eine explizite Abbruchbedingung setzen** (Rundenzahl oder Zeitbudget) — sonst entscheidet das Modell selbst, wann "fertig" ist.
3. **Getrennte Bau-/Prüf-Rollen statt Selbstabnahme** — sowohl die Vier-Augen-Inspektor-Regel als auch die Assembly-Line-Struktur bestätigen dasselbe Prinzip aus zwei unabhängigen Quellen.
4. **Autonomie stufenweise vergeben** (Vertrauensleiter) statt direkt bei Dauerläufen/proaktiven Agenten einzusteigen.

Geklärt durch gezielte Recherche (2026-08-22, per WebSearch/WebFetch + unabhängiger Gegenprüfung)

- **Trajectory-Labs/Apollo-"Widerspruch" aufgelöst — war ein Konflations-Fehler, kein echter Datenwiderspruch:** Anthropic's Blogpost vom 07.08.2026 "[Auto mode is now the default in Claude Code for Pro, Max, and Team plans](#)" beschreibt zwei getrennte Red-Team-Studien: **Apollo Research** (zweiwöchiger Pilot mit synthetischen Angriffen über drei Datensätze, Klassifikator-Miss-Rate 12 % → 7 % nach Härtung) und **Trajectory Labs** (72 reale Indirect-Prompt-Injection-Szenarien × 10 Wiederholungen = 720 Versuche gegen aktuelle Claude-/Codex-Versionen, Stand 17.07.2026 — **0 von 720** erfolgreich gegen Claude im Auto-Modus, gegenüber 5,83 % bei GPT-5.6/Codex Auto-Review und 19,03 % bei Codex Full Access). Die "7–11 %" aus [video-summary-9lyg9m8D3q0.md](#) gehören zu Apollo, nicht zu Trajectory Labs — das Video hat beide Studien fälschlich zusammengeworfen. [video-summary-gan2rEV9hJk.md](#) mit "0 von 720" war korrekt. Konfidenz: hoch (Primärquelle direkt gelesen, unabhängig gegengeprüft, exakte Zahlen bestätigt).
- **Andrew Ngs "The Batch"-Datum bestätigt — ursprüngliche Angabe war schon richtig:** [Three Key Loops for Building Great Software](#), veröffentlicht **26. Juni 2026** (direkt von der Originalseite verifiziert, exakter Titel, nennt Boris Cherny und Peter Steinberger explizit als Präguungsfiguren für "loop engineering", Drei-Loops-Framing nach Zeitskala deckt sich mit der Einordnung oben). Die frühere "~30. Juni"-Vermutung beruhte nur auf Suchergebnis-Snippets, nicht auf der echten Seite. Konfidenz: hoch.
- **Bezahlschranke für Loops präzisiert:** `/schedule` -Routines sind laut [offizieller Doku](#) wörtlich an Pro/Max/Team/Enterprise gebunden. `/goal` ([Doku](#)) und `/loop` ([Doku](#)) haben dagegen **keine** dokumentierte Plan-Sperre — nur generisches Nutzungskontingent. Der in [video-summary-u2v17HBnhh8.md](#) genannte Preis "15 €/Monat" stimmt nicht mit dem offiziellen Pro-Preis (20 \$, ca. 17 \$ bei Jahresabo) überein. Konfidenz: hoch.

Offene Fragen (weiterhin nicht abschließend geklärt)

- **Chronologie der fünf Engineering-Stufen** (Prompt 2022, Context Mitte 2025, Harness Anfang 2026, Loop Juni 2026, Graph "frisch"): eigene, unbelegte Einordnung eines einzelnen Video-Hosts, nicht unabhängig verifiziert, aber plausibel.
 - **"268 Fehler"-Zahl** im Excel-Beispiel aus [video-summary-u2v17HBnhh8.md](#): anhand der Frames nicht zuverlässig gegenlesbar — keine externe Quelle möglich, nur durch erneutes Sichten des Originalvideos zu klären.
-

Kernbotschaft

Loop Engineering ersetzt die manuelle Feedback-Schleife des Nutzers (prompten → prüfen → nachbessern) durch eine vom Agenten selbst betriebene, auf einer expliziten Spec, messbaren Erfolgskriterien, einer unabhängigen Prüfinstanz und einem harten Budget beruhende Schleife. Der Begriff selbst ist erst seit Juni 2026 (Boris Cherny, Peter Steinberger, aufgegriffen von Andrew Ng) im Umlauf und wird — je nach Quelle — unterschiedlich in eine größere Stufenleiter (Prompt/Context/Harness/Loop/Graph Engineering) eingeordnet; die genaue Beziehung zwischen "Harness" und "Loop" ist reihenübergreifend noch nicht einheitlich definiert. Praktisch am wichtigsten sind zwei Regeln, die sich in allen Quellen bestätigen: objektive Kriterien statt vager Kontrollsätze, und eine explizite Abbruchbedingung, damit eine Schleife nicht unkontrolliert Tokens und Zeit verbrennt.

Themen-Tags

Loop Engineering, `/goal`, `/loop`, `/schedule`, Boris Cherny, Peter Steinberger, Andrew Ng, Vertrauensleiter, Graph Engineering, Harness Engineering, Assembly Line, Karpathy LLM-Wiki, Agentic Coding